

Partie 01 — Prendre en main R

INF-STAR — Programmation statistique avec R

Oskar Laverny

September 16, 2026

Aix-Marseille Université

Objectifs

À la fin de cette partie

Vous saurez :

- vous repérer dans **VS Code** avec l'extension R ;
- distinguer **éditeur**, **terminal R**, **script** et **projet** ;
- créer des objets et appeler des fonctions ;
- exécuter une ligne de code depuis un script ;
- trouver de l'aide quand une commande est inconnue ;
- installer et charger un package ;
- organiser un travail de manière reproductible dès le départ.

Notre environnement de travail



R et VS Code ne sont pas la même chose

R est le langage et le logiciel qui exécute les calculs.

VS Code est l'éditeur dans lequel nous allons principalement :

- écrire et conserver le code ;
- lancer un terminal R ;
- explorer les fichiers du projet ;
- consulter l'aide et les objets ;
- travailler plus tard avec Quarto.

VS Code aide à écrire et à organiser le code, mais c'est toujours **R** qui l'exécute.

Pour le cours, il faut :

1. installer **R** ;
2. installer **Visual Studio Code** ;
3. installer l'extension **R** de *REditorSupport* ;
4. dans R, installer le package utilisé par l'éditeur pour l'autocomplétion :

```
install.packages("languageserver")
```

Cette installation n'est à faire qu'une fois.

Les zones à repérer

Dans VS Code, quatre zones seront particulièrement utiles :

VS Code

|

└─ Explorateur de fichiers → les fichiers du projet

└─ Éditeur → le code que l'on conserve

└─ Terminal R → R exécute les commandes

└─ Panneaux R / aide → objets, documentation, graphiques

Vous n'avez pas besoin de maîtriser toute l'interface dès le début.

Premier script dans VS Code

Créez un fichier `partie01.R` et écrivez :

```
2 + 3  
sqrt(16)
```

Placez le curseur sur une ligne et utilisez **Ctrl+Entrée** pour l'envoyer au terminal R.

Vous devez distinguer deux choses :

- le code reste écrit dans le fichier ;
- le résultat apparaît dans le terminal R.

Éditeur et terminal : deux rôles différents

Dans l'**éditeur** :

```
age <- 42  
age + 1
```

Dans le **terminal R**, vous verrez les commandes exécutées et leurs résultats.

Une commande testée seulement dans le terminal peut être perdue. Une commande importante doit finir dans un script.

R en quelques minutes



À quoi sert R ?

R est à la fois :

- un **langage de programmation** ;
- un environnement pour **manipuler et analyser des données** ;
- un écosystème de packages pour la statistique et la visualisation ;
- un outil adapté aux analyses reproductibles.

L'objectif du cours n'est pas de connaître tout R : il est de devenir autonome pour résoudre des problèmes concrets.

Une commande, un résultat

```
2 + 3  
sqrt(16)  
mean(c(12, 15, 18))
```

R lit une expression, l'évalue, puis affiche éventuellement un résultat.

Au début, exécutez souvent votre code. Les petites étapes sont plus faciles à comprendre et à corriger.

Créer un objet

L'opérateur d'affectation recommandé est `<-` :

```
age <- 42  
poids <- 70  
age
```

Un objet permet de donner un nom à une valeur et de la réutiliser.

```
age + 1
```

Des noms lisibles

Préférez :

```
age_patient <- 42  
tas_moyenne <- 128
```

à :

```
a <- 42  
x2 <- 128
```

Un bon nom évite d'avoir à se souvenir de ce que contient l'objet.

Script, terminal et projet



Le terminal R est utile pour :

- tester une commande ;
- inspecter rapidement un objet ;
- demander de l'aide ;
- voir les résultats du code envoyé depuis l'éditeur.

Mais ce qui n'existe **que dans le terminal** est difficile à reproduire.

Un script .R conserve les commandes importantes :

```
# analyse.R  
age <- c(34, 51, 68)  
mean(age)
```

Le script devient la trace de l'analyse.

Principe : si une commande mérite d'être refaite demain, elle mérite probablement d'être écrite dans un script.

Ouvrir un dossier comme projet de travail

Dans VS Code, ouvrez le **dossier racine** du travail plutôt qu'un fichier isolé.

Par exemple :

```
mon-projet/  
├─ data/  
├─ R/  
├─ resultats/  
└─ analyse.R
```

L'explorateur de VS Code permet alors de voir tout le projet au même endroit.

Travaillez avec des chemins relatifs au dossier du projet :

```
data/patients.csv
```

plutôt qu'avec :

```
C:/Users/nom/Desktop/mon-projet/data/patients.csv
```

Le premier chemin reste valable si le projet est déplacé sur un autre ordinateur.

À éviter : dépendre de l'espace de travail

Nous ne construirons pas le cours autour de `save.image()` ou de la restauration automatique de `.RData`.

On préfère pouvoir reconstruire les résultats à partir :

1. des données ;
2. des scripts ;
3. des paramètres explicitement écrits.

Cela rend le travail plus facile à relire, partager et reproduire.

Appeler des fonctions



Une fonction

Une fonction reçoit des arguments et retourne un résultat :

```
mean(c(10, 20, 30))  
round(3.14159, digits = 2)
```

La forme générale est :

```
nom_fonction(argument1, argument2, ...)
```

Arguments nommés

Nommer les arguments rend souvent le code plus lisible :

```
round(x = 3.14159, digits = 3)  
rnorm(n = 5, mean = 10, sd = 2)
```

L'ordre importe moins lorsque les arguments sont nommés.

Depuis R :

```
?mean  
help(mean)  
args(mean)  
example(mean)
```

L'extension R de VS Code peut aussi afficher les pages d'aide dans l'éditeur.

Sur le web, recherchez de préférence la documentation officielle du package concerné.

```
moyenne(age)
```

Si `moyenne()` n'existe pas, R vous le dit.

Réflexe :

1. lire le message jusqu'au bout ;
2. identifier la commande concernée ;
3. vérifier le nom de l'objet ou de la fonction ;
4. réduire le problème à une petite instruction.

Packages



Installer puis charger

Une installation se fait en général **une seule fois** :

```
install.packages("ggplot2")
```

Le chargement se fait à chaque nouvelle session qui en a besoin :

```
library(ggplot2)
```

Appeler une fonction sans charger le package

```
ggplot2::ggplot  
readr::read_csv
```

La notation `package::fonction` indique explicitement d'où vient la fonction.

Elle est très pratique dans un script pédagogique.

Un premier mini-workflow



Exemple

Dans partie01.R :

```
ages <- c(34, 51, 68, 42, 57)
```

```
mean(ages)
```

```
median(ages)
```

```
min(ages)
```

```
max(ages)
```

Puis :

```
hist(ages)
```

On a déjà utilisé l'éditeur, le terminal R, des objets, des vecteurs et des fonctions.

Bonnes habitudes dès maintenant

- ouvrez le **dossier du projet** dans VS Code ;
- écrivez le code important dans un script ;
- utilisez des noms d'objets explicites ;
- ajoutez quelques commentaires utiles ;
- gardez les données originales intactes ;
- évitez les chemins absolus ;
- exécutez votre script depuis une session R propre de temps en temps.

À retenir



Les cinq idées essentielles

1. **VS Code** : l'environnement dans lequel nous écrivons et organisons le travail.
2. **Terminal R** : l'endroit où R exécute les commandes.
3. **Script** : la trace conservée de ce qui doit être reproductible.
4. **Projet** : l'organisation des fichiers et des chemins.
5. **Fonctions** : la plupart des actions en R sont des appels de fonctions.

La suite du cours partira de données concrètes plutôt que de détails internes du langage.